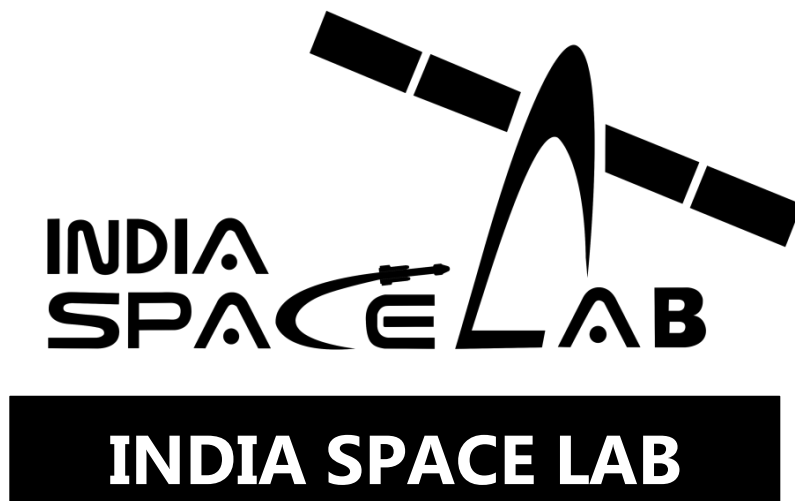


CANSAT AND CUBESAT TECHNOLOGY
REAL-TIME TELEMETRY MONITORING AND MISSION
OPERATION SOFTWARE



Project Report

01ST May to 15th June 2026

Student Name: GANGINENI RUSHIKESAV

Institute Name: JNTUA College of Engineering
Ananthapuramu

Institute Roll no: 23001A0451

Enrollment no: ISL-878154

Tables of Content

Abstract

1. Introduction
2. System Architecture
3. Graphical User Intrerface Design
4. Telemetry Processing and Data Ingestion
5. Mission Control and Fault Detection
6. Real-Time Visualization and Graphing
7. Data Logging and Preservation
8. Simulation Engine
9. Conclusion

Appendix A: System Interface (UI) Screenshots

Appendix B: Telemetry Log Excerpt

Appendix C: Exported Flight Profiles

Abstract

This document details the design, architecture, and implementation of a professional Ground Control Software (GCS) dashboard developed for a CanSat/CubeSat aerospace mission. The software operates as the primary ground station interface, engineered to ingest, decode, and visualize real-time mission telemetry. Built entirely as a client-side Single-Page Application (SPA), the system functions independently of heavy backend server frameworks. It leverages HTML5, CSS3, and ES6 Vanilla JavaScript, integrating advanced data visualization libraries to deliver a zero-latency, highly fault-tolerant operational environment.

1. Introduction

Ground Control Software is a critical component of any aerospace mission, providing ground operators with the situational awareness required to monitor vehicle health and execute commands. The objective of this project was to engineer a robust, browser-based GCS capable of interfacing via USB serial connections to receive high-frequency telemetry. The software must dynamically plot environmental data, map geographical drift, visualize payload attitude in three dimensions, and autonomously detect critical system faults during descent.

2. System Architecture

To maximize hardware compatibility and reduce computational overhead, the application architecture relies entirely on native browser engines.

- **Core Logic:** Vanilla JavaScript (ES6) is utilized for state management, asynchronous data stream parsing, and hardware interfacing.
- **Layout Engine:** A combination of CSS3 Grid and Flexbox creates a responsive, modular tactical display that scales to various monitor resolutions without distortion.
- **Visualization Stack:**
 - **Chart.js (Canvas API):** Renders high-performance, real-time rolling line graphs.
 - **Leaflet.js:** Processes geospatial data for interactive map tracking.
 - **Three.js (WebGL):** Handles 3D spatial attitude rendering and coordinate transformations.

3. Graphical User Interface (GUI) Design

The user interface is modeled after modern tactical aerospace environments, prioritizing cognitive ease and rapid data assimilation during high-stress mission phases.

- **Modular Partitioning:** The UI is segmented into distinct operational zones: Mission Commands, Fault Monitoring, Container Telemetry, Payload Telemetry, Real-Time Plotting, and Spatial Tracking.
- **Color Theory & Aesthetics:** A deep-space dark mode palette (base #04060e, panels #0a101f) is employed to significantly reduce optical fatigue. High-contrast neon cyan and green accents denote nominal operational states, while high-intensity pulsing red animations are reserved for system faults.
- **Typography:** To prevent visual layout shifts when data updates at high frequencies (1 Hz or greater), strict monospace typography (Share Tech Mono) is applied to all numerical data blocks. Structural headers utilize futuristic sans-serif fonts (Orbitron, Rajdhani) to maintain visual hierarchy.

4. Telemetry Processing and Data Ingestion

The GCS features a custom parsing engine built to decode standardized comma-separated value (CSV) telemetry arrays.

- **Packet Structure:** The software extracts 15 distinct parameters per transmission: [TEAM_ID, TIME, ALTITUDE, PRESSURE, TEMPERATURE, VOLTAGE, DESCENT_RATE, LATITUDE, LONGITUDE, SATELLITES, PITCH, ROLL, YAW, SEPARATION_STATE, PARACHUTE_STATE].
- **Hardware Interfacing:** The application utilizes the native **Web Serial API** (navigator.serial). This allows the browser to establish a direct, asynchronous USB connection with physical microcontrollers, processing the TextDecoderStream continuously.
- **Data Routing:** Parsed integers and floats are instantly injected into the Document Object Model (DOM). Environmental data (Altitude, Pressure, Temp, Battery) is routed to the Container panel, while flight dynamics (Descent Rate, IMU, GPS) are routed to the Payload panel.

5. Mission Control and Fault Detection

A primary safety feature of the GCS is its autonomous Fault Detection System, represented visually by a 4-digit LED matrix. The JavaScript state engine continuously evaluates the incoming telemetry against predefined aerodynamic and mechanical thresholds:

- **Digit 1 (Descent Rate):** Flags 1 (Active Fault) if the CanSat's descent velocity breaks outside the safe aerodynamic envelope of 8.0 m/s to 10.0 m/s.
- **Digit 2 (GPS Lock):** Flags 1 if satellite lock drops to 0, indicating a critical loss of tracking telemetry.
- **Digit 3 (Separation):** Flags 1 if the payload separation mechanism registers a failure to deploy.

- **Digit 4 (Parachute):** Flags 1 if the emergency parachute deployment sequence has been triggered. Additionally, the command interface logs all operator actions (e.g., triggering manual separation) with an exact timestamp to an onboard operational console.

6. Real-Time Visualization and Graphing

Complex numerical data is translated into live visual models to provide immediate operational feedback.

- **Rolling Line Graphs:** Five independent Chart.js canvases plot continuous timelines for Altitude, Pressure, Temperature, Descent Rate, and Battery Voltage. A **Rolling Window Algorithm** is implemented to prevent memory overflow; once an array exceeds 40 ticks, the oldest data points are dynamically shifted out (.shift()), creating a seamless scrolling effect.
- **Spatial Tracking:** Leaflet.js processes incoming geographic coordinates over an OpenStreetMap tile layer. The system utilizes the L.polyline class to draw a continuous trajectory line, mapping the payload's physical drift due to atmospheric wind shear.
- **Attitude Profiling:** A WebGL canvas powered by Three.js renders a geometric representation of the CanSat. The software converts incoming Pitch, Roll, and Yaw telemetry from degrees to radians, applying these mathematical transforms to the 3D mesh (.rotation.x, .y, .z) to visualize the payload's stabilization profile.

7. Data Logging and Preservation

Preserving flight data for post-mission analysis is executed through non-blocking asynchronous functions.

- **Telemetry Array Export:** Every raw CSV string is pushed into an internal memory array. Utilizing a Blob object with a text/csv MIME type, the software compiles this array and forces a local download of the complete telemetry_flight_log.csv.
- **Graph Rendering & Export:** Because Chart.js draws on a transparent canvas, the software programmatically generates a temporary canvas, fills it with the application's dark background hex code (#0a101f), and redraws the graph over it. The five primary flight graphs are then downloaded sequentially as high-resolution .png images, separated by a 500ms setTimeout to ensure the browser successfully processes multiple file downloads.

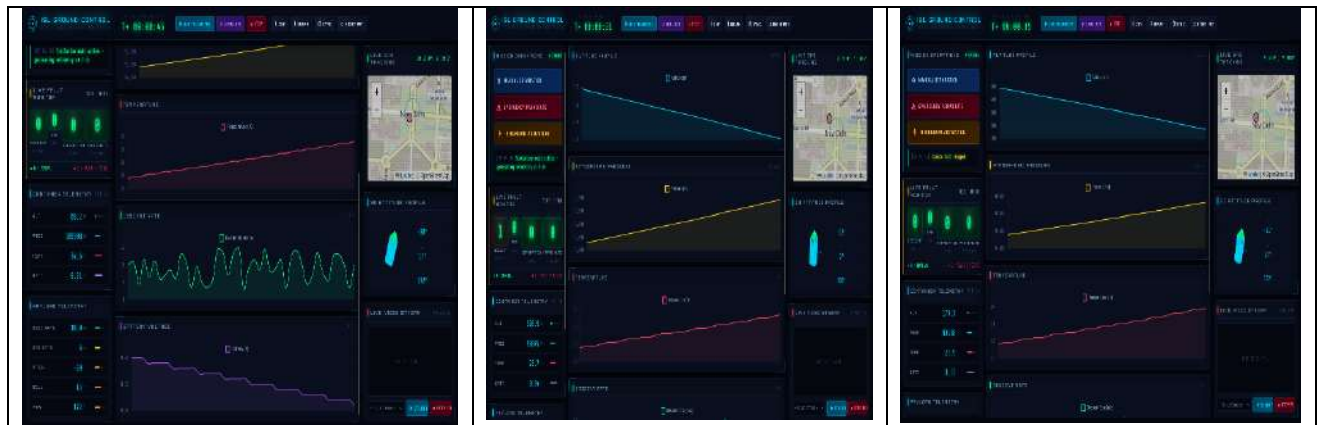
8. Simulation Engine

To validate the software ecosystem under controlled parameters without a physical telemetry transmitter, an internal mathematical simulation engine was integrated. Utilizing setInterval, the simulator models a CanSat descent starting from a 500m apogee. It continuously recalculates randomized descent variables, resultant atmospheric pressure drops, and sine-wave IMU rotations at 1 Hz, effectively proving the robustness of the UI and data-handling protocols.

9. Conclusion

The developed CanSat Ground Control Software successfully integrates modern web technologies to deliver a professional-grade telemetry monitoring system. Through the utilization of the Web Serial API, modular CSS layouts, and highly optimized JavaScript visualization algorithms, the software provides a scalable, fault-tolerant toolset capable of safely tracking and managing real-time aerospace operations.

Appendix A: System Interface (UI) Screenshots



Appendix B: Telemetry Log Excerpt

TEAM_ID	TIME	ALT	PRES	TEMP	VOLT	DESC	LAT	LNG	SATS	PITCH
1001	0	490	95576	21.8	8.4	10	28.6139	77.21	7	0
1001	1	479.8	95692	21.9	8.4	10.2	28.614	77.21	6	3.1
1001	2	470.9	95794	21.9	8.4	8.9	28.6141	77.20998	9	6.2
1001	3	460.7	95912	22	8.39	10.3	28.6142	77.20996	6	9.2
1001	4	451.4	96019	22.1	8.39	9.3	28.61429	77.20992	7	12
1001	5	441.2	96136	22.1	8.39	10.2	28.61438	77.20988	7	14.6
1001	6	432	96241	22.2	8.39	9.2	28.61446	77.20983	8	17
1001	7	422.6	96350	22.3	8.39	9.4	28.61454	77.20976	9	19.2
1001	8	412.9	96462	22.3	8.38	9.7	28.61462	77.2097	8	21
1001	9	403.8	96567	22.4	8.38	9.2	28.61468	77.20962	8	22.6
1001	10	393.2	96690	22.4	8.38	10.6	28.61474	77.20954	7	23.7
1001	11	383	96808	22.5	8.38	10.2	28.61479	77.20945	8	24.5
1001	12	374.3	96908	22.6	8.38	8.7	28.61483	77.20936	9	24.9
1001	13	364.5	97023	22.6	8.37	9.9	28.61486	77.20927	8	25
1001	14	355.5	97127	22.7	8.37	9	28.61489	77.20917	7	24.6
1001	15	346.8	97228	22.7	8.37	8.7	28.6149	77.20907	9	23.9
1001	16	337.6	97334	22.8	8.37	9.1	28.6149	77.20897	8	22.7
1001	17	326.8	97461	22.9	8.37	10.9	28.61489	77.20887	7	21.3
1001	18	316.1	97585	22.9	8.36	10.7	28.61487	77.20877	8	19.5
1001	19	306.7	97694	23	8.36	9.4	28.61485	77.20868	8	17.3
1001	20	296	97819	23.1	8.36	10.7	28.61481	77.20858	8	15
1001	21	285	97948	23.1	8.36	11	28.61476	77.2085	7	12.3
1001	22	276.5	98048	23.2	8.36	8.6	28.61471	77.20841	7	9.5
1001	23	266.5	98164	23.3	8.35	9.9	28.61465	77.20833	7	6.6
1001	24	256.2	98285	23.3	8.35	10.3	28.61458	77.20826	7	3.5
1001	25	246.5	98398	23.4	8.35	9.7	28.6145	77.2082	7	0.4
1001	26	235.7	98526	23.5	8.35	10.9	28.61442	77.20814	8	-2.7
1001	27	226.1	98638	23.5	8.35	9.6	28.61433	77.2081	7	-5.8
1001	28	216.3	98754	23.6	8.34	9.8	28.61423	77.20806	9	-8.8
1001	29	205.9	98875	23.7	8.34	10.3	28.61414	77.20803	7	-11.6
1001	30	197.3	98977	23.7	8.34	8.7	28.61404	77.20801	7	-14.3
1001	31	186.7	99102	23.8	8.34	10.5	28.61394	77.208	8	-16.7
1001	32	175.9	99229	23.9	8.34	10.8	28.61384	77.208	9	-18.9
1001	33	165.8	99349	23.9	8.33	10.1	28.61374	77.20801	9	-20.8
1001	34	156.9	99454	24	8.33	8.9	28.61364	77.20803	6	-22.4
1001	35	147.5	99565	24	8.33	9.4	28.61355	77.20806	7	-23.6
1001	36	136.8	99692	24.1	8.33	10.7	28.61346	77.2081	9	-24.4
1001	37	127.2	99807	24.2	8.33	9.6	28.61337	77.20815	8	-24.9
1001	38	118.1	99915	24.2	8.32	9.1	28.61329	77.20821	6	-25
1001	39	107.6	100039	24.3	8.32	10.5	28.61321	77.20827	9	-24.7
1001	40	97	100165	24.4	8.32	10.6	28.61314	77.20835	7	-24
1001	41	86.9	100286	24.4	8.32	10.1	28.61308	77.20843	7	-22.9
1001	42	78.2	100389	24.5	8.32	8.7	28.61303	77.20851	6	-21.5
1001	43	68.2	100508	24.6	8.31	10	28.61298	77.2086	6	-19.7
1001	44	57.3	100638	24.6	8.31	10.9	28.61295	77.20869	8	-17.6

1001	45	47.7	100753	24.7	8.31	9.6	28.61292	77.20879	0	-15.3
1001	46	37.4	100877	24.8	8.31	10.4	28.61291	77.20889	0	-12.7
1001	47	27.8	100992	24.8	8.31	9.6	28.6129	77.20899	0	-9.9
1001	48	18.7	101101	24.9	8.3	9.1	28.6129	77.20909	0	-7
1001	49	8.5	101223	24.9	8.3	10.2	28.61292	77.20919	0	-3.9
1001	50	0	101325	25	8.3	9.1	28.61294	77.20928	0	-0.8

Appendix C: Exported Flight Profiles

